

25/09/25

2) $s1 = 2$

$s2 = 20$

for i in range($s1, s2 + 1, 1$):

for j in range($2, i, 1$):

if $(i \% j == 0)$:

break

else:

print(i)

without function

o/p:-

2

3

5

7

11

13

17

19

Step 1: Outer loop (for i in range(81, 82+1, 1))

• this loop sets i to all numbers b/w 81 and 82.

So first $i=2$ then $i=3$ then $i=4$... up to $i=80$

Step 2: Inner loop (for j in range(2, i, 1))

• For each i , we check if it has any divisors

• j goes from 2 to $i-1$

Step 3:

if $C : i \% j == 0$:

break

• $\%$ means remainder

• if $i \% j == 0$, that means i is divisible by $j \rightarrow$ so i is not Prime

• break will stop the inner loop immediately.

Step 4: The special else of for

In Python, for... else means:

• The else runs only if the loop finishes without a break.

• In our case: if no divisor was found, then else runs $\rightarrow$ we print i .

Let's trace from no

Case $i=2$

• Inner loop: range(2, 2) $\rightarrow$ empty (no j values)

• So loop doesn't run at all, and it didn't break.

• else runs $\rightarrow$ prints 2.

Case $i=4$

• Inner loop: $j=2$ only

• checks: $4 \% 2 == 0 \rightarrow$ true

• So break happens $\rightarrow$ inner loop stops

• Because we used break, else does not run.

• So 4 is not printed.

Case $i=5$

• inner loop: $j=2, 3, 4$

$j=2 \quad 3 \quad 4$

• None of them divide 5 evenly $\rightarrow$ no break

• Loop ends normally $\rightarrow$ else runs $\rightarrow$ prints 5

Using function Print Prime no Program

```
def prime1(r1, r2):  
    for i in range(r1, r2 + 1, 1):  
        for j in range(2, i, 1):  
            if (i % j == 0):  
                break  
            else:  
                print(i)
```

~~def~~ calling the function
~~prime1(1, 10)~~
prime1(1, 10)

o/p :
2
3
5
7

→ def prime1(r1, r2):

- Defines a function named prime1 with two inputs:
r1 (start) and r2 (end)

→ for i in range(r1, r2 + 1, 1):

- Outer loop: takes each integer from r1 up to and including r2.

- Each i is a candidate number we will test for primality.

→ for j in range(2, i, 1):

- inner loop: j runs from 2 to i-1. These are potential divisors of i.

- if i has any divisor j in this range (i % j == 0), then i is not prime.

→ if (i % j == 0):

- checks if j divides i exactly (remainder zero).

→ break

- if a divisor is found, break stops the inner loop immediately.

- else: (this else paired with the inner loop for...)
- important: in Python for... else means the else block runs only if the loop finished normally (i.e. no break happened)
- So, else: print(i) runs only when we didn't find any divisor - meaning i is prime - so we print it.
- i=2 → inner loop range(2,2) is empty → no break → else prints 2
- i=3 → inner loop j=2 → 3%2 != 0 → loop finishes → else prints 3
- i=4 → inner loop j=2 → 4%2 == 0 → break → else skipped → no print
- i=5 → check j=2,3,4 → none divide 5 → else prints 5
- i=6 → j=2 → 6%2 == 0 → break → skip

Outer loop @

→ Take each no one by one from n1 to n2.

Inner loop

• for dividing i by all no from 2 up to i-1

if divisible @

- if any no divides i exactly, it means i is not prime → stop checking (break)
- Else (after inner loop):
if no no divided i, then i is prime → print it

Ex: prime(8,10)

- i=2: no no to test → print 2
- i=3: test j=2 → not divisible → print 3
- i=4: test j=2 → divisible → don't print

def prime(n1, n2):

l = [] - makes an empty list to store prime no

for i in range(n1, n2+1, 1):

for j in range(2, i, 1):

if (i%j == 0):

break

else:

1. append(i) - when the program finds a prime no,
it puts that prime into the list 'l'
return l

calling the func

Prime(2, 10)

[2, 3, 5, 7]

2) Create a func with 3 args and return to find largest among 3 numbers.

```
def lar(a, b, c):  
    if (a > b and a > c):  
        return f"{a} is large"  
    elif (b > a and b > c):  
        return f"{b} is large"  
    else:  
        return f"{c} is large"
```

calling the func
lar(12, 99, 155)

o/p: '155 is large'

Note: Another type of function in Python

Function as a Parameter

We can assign them to variables

We can pass them as parameters to other functions

We can return them from functions

```
def Square(x):  
    return x * x
```

```
def cube(x):  
    return x * x * x
```

```
def apply-function(fun-name, num):  
    return fun-name(num)
```

calling: apply-function(Square, 5)
o/p: 25

- Functions (Square and cube) are standalone functions.
- apply - function takes another function as %p & runs it.
- ~~Ex:~~ is called passing a funct as a parameter, not a nested function.

Nested function Exmp. outer func

```
def apply_function(num, power):
    # nested function defined inside.
    def square(x):
        # in new func
        return x * x
    def cube(x):
        return x * x * x
    if power == 2:
        return square(num) # call the inner func square.
    elif power == 3:
        return cube(num) # call the inner func cube.
```

```
def outer():
    print("I am outer")
    def inner():
        print("I am inner")
    inner() # call inner function inside outer
# call outer function
outer()
```

Note:- Without calling outer() func U can't call the inner() func means U can't get output.

Inner func runs only when outer runs

outer func = big box

Inner func. Small box inside.

U can't open small box inside. without opening big box

```
def outer():  
    print("I am outer")
```

```
def inner():  
    print("I am inner")
```

```
    return inner # return inner funct without running it
```

```
# call outer & get inner funct
```

```
my_func = outer() # prints "I am outer"
```

```
# call inner function later
```

```
my_func() # prints "I am inner"
```

- outer funct

- prints "I am outer"

- Defines inner funct inner()

- Returns inner without calling it

- Call outer:

- my_func = outer()

- Runs outer → prints "I am outer"

- Returns inner → stored in my_func

- call inner later:

- ~~my_func~~ my_func()

- Runs inner → prints "I am inner"

o/p: I am outer

I am inner

- outer funct runs first.

- Inner funct is returned & can be run later.

- This shows how nested funct can be stored & called separately

Diagram : Outer returns inner

call: my_func = outer()

outer func runs

Prints: "I am outer"

Inner func is created

& returned → stored in

my_func

call: my_func()

Inner func runs

Prints: "I am inner"

Note: inner func can access outer func variables

def outer():

x = 10 # outer variable

def inner():

print(x) # inner can see outer variable

inner()

outer()

o/p: 10

variables defined in the outer func are visible inside the inner func

∴ Outer func cannot access inner func variables

def outer():

def inner():

y = 20 # inner variable

inner()

print(y) # X Error: y not accessible here

outer()

Variables defined inside the inner func are not visible outside.

Recursion Function:-

a recursive function is a function that call itself until a base condition met.

def fact(n):

if n==1:

return 1

else:

return n * fact(n-1)

calling fact(5)

o/p: 120

Base case stops the recursion when n becomes 1, it returns 1

Recursive case

for n>1, the func call itself with n-1

• then multiplies the result by n

• this repeats until it reaches the base case

~~5 * 4 * 3 * 2 * 1~~

Nested Funct

Syntax:-

def outer_fun(p1, p2, ..., pn):

def inner_fun(p1, p2, ..., pn):

return value

return value

Ex:-

def outer_fun(num):

Print("outer received:", num)

def inner_fun(x):

return x * 2

return inner_fun(num)

result = outer_fun(5)

Print("Result:", result)

outer funct → defines inner funct

inner funct → does actual calculation

Outer calls inner & returning the result

lambda function:

- * a lambda function is a small, anonymous function in Python.
- * it was defined using the keyword lambda instead of def
- * it can take any number of arguments but must contain only one expression.
- * expression is automatically returned.
(no need to use return)

Syntax:-

lambda arguments : expression.

g = lambda num : num * num

calling g(5)

o/p 25

a = lambda num1, num2 : num1 + num2

a(5, 2)

7

Nested lambda:

outer = lambda x: (lambda y: y * y)(x + 5)

outer(3)

1) Outer lambda

lambda x: (---)

• takes i/p x $\rightarrow$ in this example $x = 3$

2) Inner lambda (lambda y: y * y)

• takes i/p y $\rightarrow$ squares it ($y * y$)

3) How it works

$(x + 5) \rightarrow 3 + 5 = 8$

lambda y: $y * y \rightarrow 8 * 8 = 64$

4) Final result: 64